POS DIGITAL
GETNET

CLASSIFICAÇÃO DO DOCUMENTO – CONFIDENCIAL

Este documento pode conter informação confidencial e/ou privilegiada. Se você não for o destinatário ou a pessoa autorizada a receber este documento, não deverá utilizar, copiar, alterar, divulgar a informação nele contida ou tomar qualquer ação baseada nessas informações. Se você recebeu este documento por engano, por favor, avise imediatamente o responsável, devolvendo o documento em seguida. Agradecemos sua cooperação.

Sumário

Histórico de alterações 4
Introdução 4
Equipamentos disponíveis 5
Gertec - GPOS700 5
Ingenico APOS A8 7
Solicitação de equipamentos 8
O que você precisa saber antes de começar a desenvolver 9
Requisitos mandatórios 9
Recomendações 9
Submetendo um Aplicativo 10
Processo de Homologação 10
Análise de Aceitação Técnica 12
Sistema de priorização 12
Teste Completo 12
Relatório dos Testes 14
Defeitos 14
Defeitos de API 14
Melhorias 14
API de Pagamento 15
Simulador da API 15
Integração com a API de pagamento 15
Deeplinks disponíveis 16
Resultados 16
Funcionalidades 16
Pagamento 16
Pré-autorização 22
Estorno 24
Reimpressão 25
Info 26
SDK 28
Configurar Gradle 28

Fazendo bind no service 29
Card 31
Impressora 32
Mifare 33
Leds 34
Beeper 34
Camera 34
Info 36
Estatística 36
Estatística Geral 37
Estatística por App 37
Dúvidas Frequentes 38

Histórico de alterações

Versão	Data	Autor	Descrição
1.0	26/04/2018	Vinicius P. Bohrer	Versão inicial
2.0	30/05/2018	Felipe Espitalher dos Passos	Mais campos no payment
2.1	21/06/2018	Vinicius P. Bohrer	Mais campos no refund e payment Mais um requisito mandatório
2.2	09/07/2018	Luiz Gustavo	Ajuste no campo **type** em payment
API 2.2 SDK 1.0	31/10/2018	Vinicius Pessil Bohrer	Processo de homologação dos apps Utilização do SDK

Introdução

O objetivo deste documento é apresentar tecnicamente como integrar aplicativos Android com a plataforma de pagamento da GETNET, assim como, requisitos de software e hardware, boas práticas de desenvolvimento e a forma de submeter o aplicativo para certificação e publicação.

Essa especificação pode ser alterada no futuro e a Getnet poderá solicitar que o aplicativo seja adaptado, principalmente com o surgimento de novos hardwares.

Juntamente com este documento, você terá acesso também aos arquivos abaixo:

Rebatedor.apk: aplicativo que simula as chamadas (deeplinks) de pagamento.

servicePosDigital: é o serviço do SDK que fica rodando no terminal que se comunica com os recursos de hardware;

libPosDigital: é a interface do SDK que deve ser compilado com o seu app para se comunicar com o serviço

Devkit: é um app de ajuda que acessa os serviços e tem alguns exemplos. Com ele, você consegue validar se os serviços de hardware estão funcionando corretamente.

Equipamentos disponíveis

Gertec - GPOS700



Fabricante	Gertec
Nome	GPOS700
Processador	QuadCore 1,2 GHz
Memória	SD RAM: 1 GB FLASH ROM: 8 GB Memória Externa: MicroSD (até 64GB)
Sistema Operacional	Android 5.1 (22)
SIM	1 SIM
SAM	2 SAM
Interface	Magnética: Bidirecional Trilhas 1, 2 e 3 - ISO 7810 e 7811 Smart Card: ISO 7816 EMV200/POBC3.0 L1/L2 Contactless / NFC: ISO 14443 Tipo A/B, Cartão Mifare® e Felica Frequência de trabalho 13.56MHz
Display	5,5"

Resolução	1280 x 720 pixels
Densidade	240 dpi (HDPI)
Teclado	Teclado virtual
Conectividade	Wi-Fi: 802.11 b/g/n com suporte a 2,4G/5G 2G: GSM/EDGE 850/ 900/ 1800/ 1900MHz 3G: WCDMA 850/900/1800/2100 MHz 4G: LTE FDD 1/3/7 e TDD 38/39/40/41 Bluetooth: 4.0
Bateria / Carregamento	Bateria - 3,7V 6000mA Adaptador CA - 5,5V, 2 Micro-USB
GPS	Sim
Câmera	Câmera Traseira - 5M pixel com zoom automático Código de Barras / QRCode: Leitor via câmera (1D/2D)
Impressora	Tipo - Térmica Velocidade - High speed Diâmetro da bobina - Ø 30 mm
Dimensões	206 x 84 x 57 mm
Peso	490 g
Sobre	https://www.gertec.com.br/produtos/gpos700/

Ingenico APOS A8





Fabricante	Ingenico
Nome	APOS A8
Processador	ARM Cortex-A7 Quad Core ARM Cortex-M4 32-bit
Memória	1GB RAM+ 8GB FLASH; Opcional 2GB RAM + 16GB Flash
Sistema Operacional	Android 5.1 (22)
SIM	1 SIM
SAM	2 SAM
Interface	Tarja Magnética: ISO 1/2/3 Smart Card: EMV Level 1 Contactless: EMV LEVEL 1&2 compliant
Display	5,5"
Resolução	1280 x 720 pixels
Densidade	320 dpi
Teclado	Teclado virtual

Conectividade	4G cat4 / 3G / 2G + WiFi bgn WiFi bgn BT 3.x
Bateria / Carregamento	2600 mAh x 2 Carregador 1A
GPS	Sim
Câmera	Câmera Traseira - 5M pixel com zoom automático Código de Barras / QRCode: Leitor via câmera (1D/2D)
Áudio	1 auto-falante 1 entrada de fone de ouvido 1 Microfone
Impressora	Tipo - Térmica Velocidade - High speed Diâmetro da bobina - Ø 40 mm
Dimensões	183 × 84 × 64 mm
Peso	450g
Sobre	https://www.ingenico.com.br/smart-terminals/pos/android/apos.html

Solicitação de equipamentos

Se você ainda não possui nenhum terminal, preencha o https://pt.surveymonkey.com/r/MS6CRBX.

Importante: O equipamento poderá ficar em posse do desenvolvedor por 90 dias. Após esse período, deve ser devolvido à Getnet.

O que você precisa saber antes de começar a desenvolver

- O desenvolvimento do aplicativo pode ser nativo ou híbrido (Exemplo: Xamarin, porque o Xamarin irá compilar o código para o “Android Runtime (ART)” como se fosse um app nativo)
- É expressamente proibido WebApps e WebView devido a medidas de segurança
- O GooglePlay Services não estará disponível
- Suporte ao Android 5.1 (22)
- É de responsabilidade do desenvolvedor garantir a segurança da informação circulada dentro do aplicativo.

- A homologação da Getnet é criteriosa e considera as questões de segurança, como também os itens mandatórios e recomendados.

Requisitos mandatórios

- O aplicativo deve possuir uma seção de "Fale Conosco" informando ao cliente os contatos necessários para atendimento em caso de problemas ou dúvidas. Essa seção deve estar bem visível e de fácil acesso.
- O aplicativo deve utilizar o número de série do equipamento disponibilizado pela API da Getnet, portanto, não é permitido usar o número que o Android disponibiliza, pois não será o mesmo que o da etiqueta colada no verso do aparelho.
- Ícone/logo – o aplicativo deve ter ícone/logo e não pode estar vinculado às identificações visuais dos itens abaixo:
    - Getnet e seus produtos;
    - Banco Santander
    - Concorrentes diretos ou indiretos da Getnet ou do grupo Santander (incluindo bancos, adquirentes, subadquirentes e wallets)
- O aplicativo não pode se comunicar com nenhum concorrente direto ou indireto da Getnet ou do Grupo Santander (incluindo bancos, adquirentes, subadquirentes e wallets)
- Definição da Política de privacidade

Recomendações

- Uso consciente do consumo de dados

- Uso consciente do consumo de bobina de impressão – evite imagens grandes, deixar grandes espaços entre linhas e espaços no início dos comprovantes.
- Uso consciente do consumo de memória e cpu
- Utilização das boas práticas de programação Android
- Evite funcionalidades com timeout infinito que possa deixar sua aplicação travada aguardando algum evento.
- Utilizar layout dinâmicos considerando que no futuro o app pode ser instalado em outros hardwares com resolução e densidade diferentes.
- Tamanho do APK – revise o seu projeto e exclua imagens e dados que não são utilizados
- Evite expor dados sensíveis dos clientes, tais como, senhas, informações pessoais...

Submetendo um Aplicativo

Crie um usuário em https://getstore.getnet.com.br/developer e faça o upload do seu app.

O Envio da documentação deve ser feito através do e-mail parceiros_posdigital@getnet.com.br.

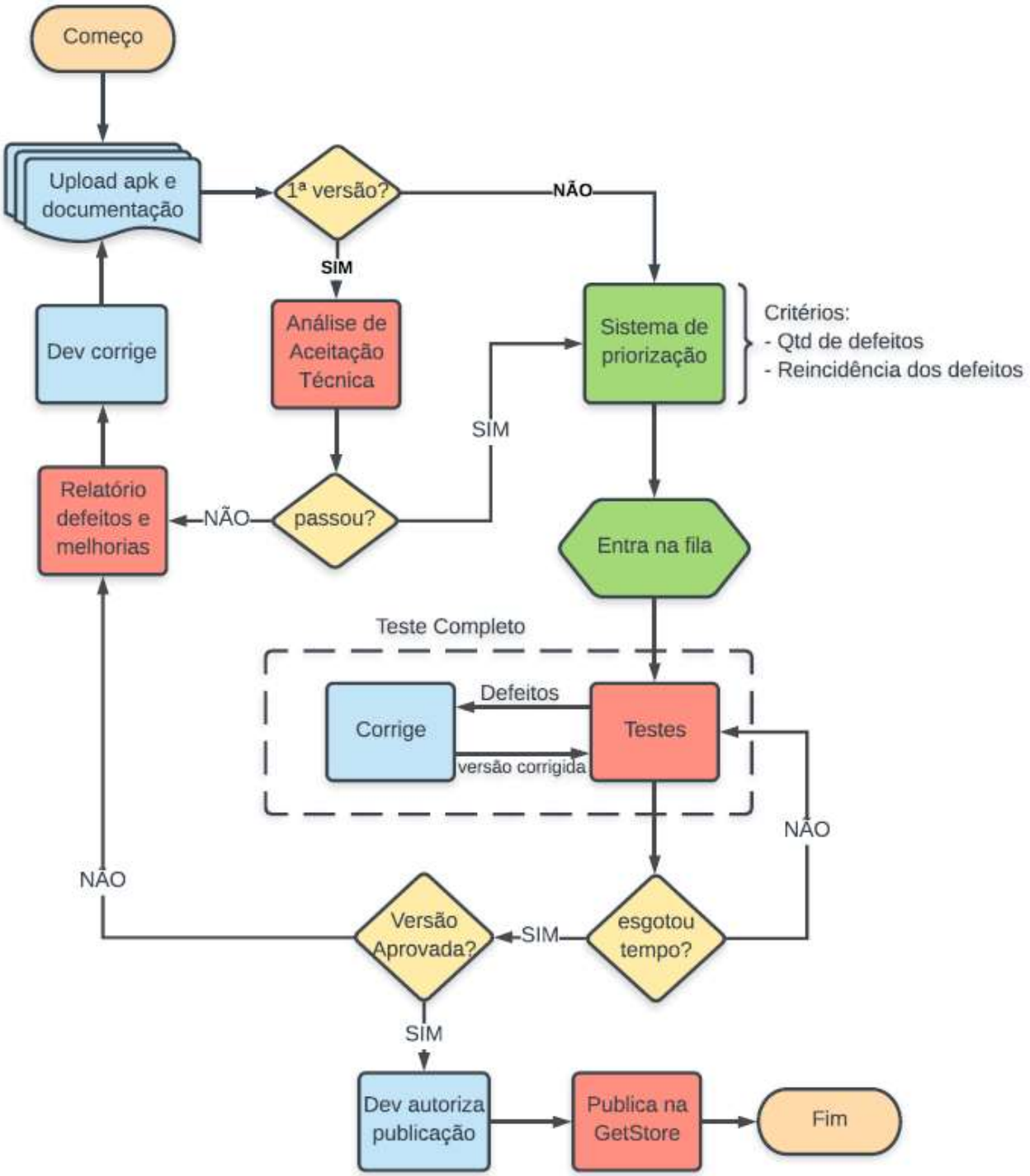
- Documentação
  - Fluxo de telas e informações das funcionalidades do app
  - Dados necessários para os testes (Ex.: credencias de acesso)
  - ReleaseNotes: para que a Getnet possa avaliar o escopo dos testes.
  - Quais recursos externos o aplicativo utiliza e os seus respectivos IPs/URLs (Ex. Crashlytics, Analytics, Firebase, servidores, ...).
  - Contatos para suporte ao aplicativo

IMPORTANTE: Não serão aceitos aplicativos que não sejam release e com conteúdo "em construção".

IMPORTANTE: O apk submetido sempre deve ter o *versioncode* maior que o anterior, mesmo se for uma versão de correção.

Processo de Homologação

Considere o fluxo abaixo para entender como funciona o processo de homologação dos apps submetidos à Getnet.


Começo
Upload apk e documentação
1ª versão?
NÃO
SIM
Dev corrige
Análise de Aceitação Técnica
Sistema de priorização
Critérios:
- Qtd de defeitos
- Reincidência dos defeitos
SIM
Relatório defeitos e melhorias
NÃO
passou?
Entra na fila
Teste Completo
Defeitos
Corrige
Testes
versão corrigida
NÃO
NÃO
Versão Aprovada?
SIM
esgotou tempo?
SIM
Dev autoriza publicação
Publica na GetStore
Fim

Análise de Aceitação Técnica

Assim que o seu aplicativo é enviado, ele passará por uma pré-análise a fim de verificar se o app atende alguns requisitos básicos e mandatórios.

Este passo deve ser rápido para que o desenvolvedor já tenha uma ideia se o seu app será suportado e aceito nos terminais da Getnet.

O aplicativo só passa deste estado até que todos os itens abaixo sejam verificados:

1) VersionCode
   - Deve ser maior que o apk anterior
2) Apk Release
   - Deve conter ícone e está assinado e em release
   - Opção de “Fale conosco”
   - Política de Privacidade
3) Permissões no manifest
   - Alguns tipos de permissões não são aceitos por permitirem que o app acesse alguns recursos do android e/ou de hardware, ou também por colocar em risco a operação de outros aplicativos. Alguns exemplos: set_time, system_alert_manager, write_apn_settings, change_network_state, kill_background_processes
4) Scan de segurança: webview e vulnerabilidades
5) Documentação: telas, credenciais para testes, release notes, servidores, contato...

IMPORTANTE: mesmo que o aplicativo passe por esta etapa, não significa que no teste completo não sejam encontradas irregularidades.

Sistema de priorização

Todo aplicativo recebido entra na fila para testes com uma determinada prioridade. Esta priorização é determinada pelo relatório da última homologação efetuada, considerando a quantidade de defeitos e defeitos reabertos. Portanto, por exemplo, se no último relatório, foram detectados muitos defeitos e ainda alguns não foram corrigidos, a prioridade diminui.

Teste Completo

Nesta etapa, os testadores da Getnet irão focar nos testes do aplicativo por um tempo determinado de acordo com quantidade de funcionalidades do app. Durante este período, para

agilizar, os testadores poderão adiantar os defeitos/melhorias detectados para que o desenvolvedor posso corrigir em tempo para não perder a janela de testes. Após o tempo determinado (ou assim que finalizar), será gerado um relatório indicando se a versão foi aprovada ou rejeitada. O Desenvolvedor indica se deseja que a versão seja publicada e atualizada nos terminais, se for o caso.

IMPORTANTE: Os testes realizados do aplicativo submetido são bastante rigorosos. O plano de testes não abrange somente a integração com a aplicação de pagamento, mas também verifica questões de segurança e usabilidade. O testador navega por todas as telas do aplicativo. Contudo, dependendo da situação encontrada, o testador irá reportar como defeito ou melhoria.

A aprovação do aplicativo não irá acontecer até que todos os itens abaixo sejam atendidos:

- **Aplicativo completo Release**, as funcionalidades do app devem estar de acordo com a documentação, não podem possuir conteúdo "em construção" e o app não pode estar em modo demonstração, ou seja, deve estar próximo do produtivo.
    - Deve ser possível navegar por todas telas sem o aplicativo "quebrar"
- **Questões de segurança**
    - É expressamente proibido WebApps e Webviews devido a medidas de segurança
    - Acesso a recursos externos impróprios
    - Integridade dos dados
    - Quaisquer itens que a GETNET julgar uma violação de segurança
- Requisitos mandatórios
- **Uso abusivo de recursos** (Bateria, Impressão, 3G e memória)
    - O app deve funcionar majoritariamente off-line para evitar gastos excessivos de dados e instabilidade na operação. Não são aceitas as reproduções de vídeo, fotos de alta qualidade e áudios longos com uso de dados.
    - Fichas, comprovantes e Notas devem ser impressas sem uso excessivos de espaços em branco e imagens grandes. A impressão desnecessária é requisito para não aprovação de APK.
- **Integração com a API de pagamento** - a integração deve estar de acordo com a especificação e o aplicativo não pode "quebrar". Deve ser possível efetuar pagamento e estorno. O aplicativo deve conseguir interpretar os retornos da API e apresentar de forma coerente em tela.
- **Suporte ao Android 5.1 (22)**
- **Usabilidade e clareza das informações –** para o estabelecimento e o portador do cartão, é importante que certas informações estejam claras na tela, tais como, valor, número de parcelas, produtos, forma de pagamento, etc. O aplicativo não pode travar, fechar abruptamente e trocar de telas erroneamente.

Relatório dos Testes

Será enviado por e-mail uma planilha excel com os itens encontrados divididos em 3 categorias, defeito, defeito de API e melhoria.

Defeitos

São todos os erros encontrados que impactam somente o aplicativo. Normalmente, são problema de usabilidade ou telas que estão quebrando. Este tipo de defeito enfraquece muito a prioridade do aplicativo quando ele for submetido novamente para homologação, pois a Getnet considera que este tipo de defeito independe do POS Digital e poderia ter sido detectado na homologação do próprio desenvolvedor. Dependendo da gravidade do defeito, a Getnet poderá aprovar a versão se o desenvolvedor sinalizar formalmente (por e-mail) que aceita os riscos do problema reportado.

Defeitos de API

São todos os erros encontrados sobre a integração do aplicativo com o POS Digital da Getnet, ou seja, erros de API, SDK e recursos de hardware. É o tipo de erro que a Getnet considera acontecer com mais probabilidade, pois é algo novo para o desenvolvedor. Precisam ser corrigidos para aprovação da versão.

Melhorias

São todos as situações que os testadores reportam sobre a usabilidade do aplicativo. A Getnet não considera que a melhoria precisa ser desenvolvida para aprovar o aplicativo.

API de Pagamento

Simulador da API

Juntamente com esta documentação, a GETNET provê um apk chamado Rebatedor.apk:

Este aplicativo simula as requisições/respostas da aplicação de pagamento, o objetivo é proporcionar para o desenvolvedor um ambiente o mais próximo possível do app Pagamento. Este simulador não irá realizar as transações online com cartões, todas as funcionalidades são "não reais" e offline - simuladas.

Integração com a API de pagamento

A integração com a API de pagamento se dará através de deeplinks parametrizados, por exemplo:

```
public class MainActivity extends AppCompatActivity {

    private final int REQUEST_CODE = 10001;

    private void startIntent(Bundle bundle, String deeplink) {
        Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse(deeplink));
        if (bundle != null) {
            intent.putExtras(bundle);
        }
        startActivityForResult(intent, REQUEST_CODE);
    }

    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (RESULT_OK == resultCode && REQUEST_CODE == requestCode ) {
            foo();
        }
    }
}
```

Deeplinks disponíveis

Todos os parâmetros deverão ser enviados através do Bundle do Intent e como String

URL	Função
getnet://pagamento/v1/payment	Pagamento
getnet://pagamento/v1/pre-authorization	Pré-autorização
getnet://pagamento/v1/refund	Reembolso
getnet://pagamento/v1/reprint	Reimpressão do último comprovante
getnet://pagamento/v1/getinfos	Retorna informações do terminal

Resultados

Toda funcionalidade retornará no final um resultado:

CÓDIGO	NOME	DESCRIÇÃO
0	SUCESSO	Transação efetuada com sucesso
1	NEGADA	Transação negada pelo servidor (emissor, bandeira, ...)
2	CANCELADA	Transação cancelada (pelo servidor ou usuário)
3	FALHA	Falha ao completar a transação (internet, servidor, emissor, ...)
4	DESCONHECIDO	Erro desconhecido

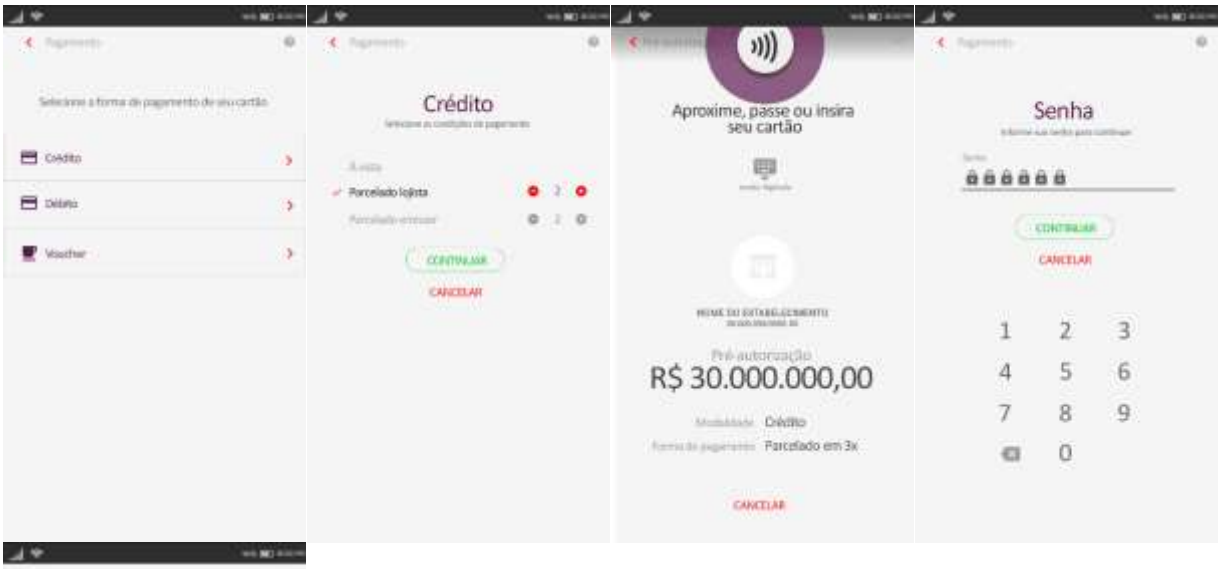
Funcionalidades

A seguir será especificado a API para cada funcionalidade disponível.

Pagamento

O deeplink **getnet://pagamento/v1/payment** irá iniciar a aplicação de pagamento da Getnet que irá tomar conta do terminal e iniciará o fluxo de pagamento solicitando primeiramente

a forma de pagamento com ilustra nas imagens abaixo. Se os dados já forem enviados no deeplink, então as telas serão automaticamente “puladas”.





No final da transação de pagamento, se a transação for aprovada, irá apresentar a tela abaixo e a impressão da via do estabelecimento iniciará automaticamente. Ao clicar em qualquer uma das duas opções, irá apresentar a mensagem de "Remova seu cartão" e a aplicação de pagamento irá sair somente após a remoção do cartão, com exceção das vendas com tarja magnética.



Caso ocorra alguma falha na transação, irá apresentar a tela abaixo e a aplicação da Getnet irá aguardar a remoção do cartão para sair.



Abaixo seguem as tabelas de requisição e resposta e seus respectivos parâmetros.

Requisição		
	Parâmetro	**Descrição**
OPCIONAL	paymentType	Este parâmetro informa qual o tipo de pagamento que será feito: **“credit”, “debit”, “voucher”** Se esse campo for informado a tela de seleção de tipo de pagamento será pulada.
OPCIONAL	creditType	Se informado no parâmetro “paymentType” crédito então este parâmetro deverá informar qual tipo de crédito: “creditMerchant” - Crédito parcelado Lojista “creditIssuer” - Crédito parcelado Emissor Se selecionado crédito e este campo for informado então a tela de tipo e crédito será pulada. Crédito à vista não precisa enviar este campo.
OPCIONAL	installments	Este parâmetro informa o número de parcelas. Se informado o parâmetro “creditType” com o valor “creditMerchant” ou “creditIssuer” então este parâmetro deve ser informado.
OPCIONAL	amount	12 dígitos representando o valor, considerando os últimos 2 dígitos como casas decimais. Exemplo: "000000001234" é equivalente R$ 12,34 Se não informado o usuário precisará preencher o valor no app de Pagamento
OPCIONAL	currencyPosition	"CURRENCY_AFTER_AMOUNT" ou "CURRENCY_BEFORE_AMOUNT" **Default : “CURRENCY_BEFORE_AMOUNT”**

OPCIONAL	currencyCode	Código da moeda, de acordo com a ISO-4217 Exemplo: "986" este é código do Real (R$) Para a lista completa das moedas acesse: https://pt.wikipedia.org/wiki/ISO_4217 **Default: "986"**
Resposta		
Quando retorna?	**Parâmetro**	**Descrição**
SEMPRE	result	Resultado da transação - para mais informações consulte a legenda deste campo
OPCIONAL	resultDetails	Texto com detalhes do retorno
SEMPRE	amount	12 dígitos representando o valor, considerando os últimos 2 dígitos como casas decimais. Exemplo: 000000001234 = R$ 12,34
OPCIONAL	nsu	Código de autorização da transação da Getnet – ele é único por terminal
OPCIONAL	cvNumber	Número do CV – informação que deve ser enviada no estorno - ele é único por terminal
SEMPRE	type	02 - Débito 11 - Crédito a vista 12 - Crédito parcelado Lojista 13 - Crédito parcelado Emissor 03 - Voucher
OPCIONAL	brand	Bandeira do cartão utilizado
SEMPRE	inputType	021 - tarja magnética 051 - chip 071 - chip sem contato 801 - tarja magnética - fallback
OPCIONAL	installments	Quantidade de parcelas selecionada

Exemplo de implementação

```
public class MainActivity extends AppCompatActivity {

    private final int REQUEST_CODE = 1001;
    private final String ARG_RESULT = "result";
    private final String ARG_RESULT_DETAILS = "resultDetails";
    private final String ARG_AMOUNT = "amount";
    private final String ARG_TYPE = "type";
    private final String ARG_INPUT_TYPE = "inputType";
    private final String ARG_INSTALLMENTS = "installments";
    private final String ARG_NSU = "nsu";
    private final String ARG_BRAND = "brand";

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Bundle bundle = new Bundle();
        bundle.putString("amount", "000000001000");
        bundle.putString("currencyPosition", "CURRENCY_AFTER_AMOUNT");
        bundle.putString("currencyCode", "986");
        Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("getnet://pagamento/v1/payment"));
        intent.putExtras(bundle);
        startActivityForResult(intent, REQUEST_CODE);
    }

    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (REQUEST_CODE == requestCode && RESULT_OK == resultCode) {
            String result = data.getString(ARG_RESULT);
            String resultDetails = data.getString(ARG_RESULT_DETAILS);
            String amount = data.getString(ARG_AMOUNT);
            String type= data.getString(ARG_TYPE);
            String inputType= data.getString(ARG_INPUT_TYPE);
            String installments= data.getString(ARG_INSTALLMENTS);
            String nsu= data.getString(ARG_NSU);
            String brand= data.getString(ARG_BRAND);
        }
    }
}
```

Pré-autorização

O deeplink **getnet://pagamento/v1/pre-authorization** irá iniciar o app de Pagamento na tela de Pré-autorização. Segue as tabelas de requisição/resposta e seus respectivos parâmetros.

Requisição		
	Parâmetro	**Descrição**
MANDATÓRIO	amount	12 dígitos representando o valor, considerando os últimos 2 dígitos como casas decimais. Exemplo: "000000001234" é equivalente R$ 12,34
MANDATÓRIO	currencyPosition	"CURRENCY_AFTER_AMOUNT" ou "CURRENCY_BEFORE_AMOUNT"
MANDATÓRIO	currencyCode	Código da moeda, de acordo com a ISO-4217 Exemplo: "986" este é código do Real (R$) Para a lista completa das moedas acesse: https://pt.wikipedia.org/wiki/ISO_4217
Resposta		
Quando retorna?	**Parâmetro**	**Descrição**
SEMPRE	result	Resultado da transação - para mais informações consulte a legenda deste campo
OPCIONAL	resultDetails	Texto com detalhes do retorno
SEMPRE	amount	12 dígitos representando o valor, considerando os últimos 2 dígitos como casas decimais. Exemplo: 000000001234 = R$ 12,34
OPCIONAL	nsu	Código de autorização da transação
SEMPRE	type	11 - Crédito a vista 12 - Crédito parcelado Lojista
OPCIONAL	brand	Bandeira do cartão utilizado
SEMPRE	inputType	021 - tarja magnética 051 - chip 071 - chip sem contato 801 - tarja magnética - fallback
OPCIONAL	installments	Quantidade de parcelas selecionada

Exemplo de implementação

```
public class MainActivity extends AppCompatActivity {

    private final int REQUEST_CODE = 1001;
    private final String ARG_RESULT = "result";
    private final String ARG_RESULT_DETAILS = "resultDetails";
    private final String ARG_AMOUNT = "amount";
    private final String ARG_TYPE = "type";
    private final String ARG_INPUT_TYPE = "inputType";
    private final String ARG_INSTALLMENTS = "installments";
    private final String ARG_NSU = "nsu";
    private final String ARG_BRAND = "brand";

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Bundle bundle = new Bundle();
        bundle.putString("amount", "000000001000");
        bundle.putString("currencyPosition", "CURRENCY_AFTER_AMOUNT");
        bundle.putString("currencyCode", "986");
        Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("getnet://pagamento/v1/pre-authorization"));
        intent.putExtras(bundle);
        startActivityForResult(intent, REQUEST_CODE);
    }

    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (REQUEST_CODE == requestCode && RESULT_OK == resultCode) {
            String result = data.getString(ARG_RESULT);
            String resultDetails = data.getString(ARG_RESULT_DETAILS);
            String amount = data.getString(ARG_AMOUNT);
            String type= data.getString(ARG_TYPE);
            String inputType= data.getString(ARG_INPUT_TYPE);
            String installments= data.getString(ARG_INSTALLMENTS);
            String nsu= data.getString(ARG_NSU);
            String brand= data.getString(ARG_BRAND);
        }
    }
}
```

Estorno

O deeplink **getnet://pagamento/v1/refund** irá iniciar o app de Pagamento na tela de estorno. Segue as tabelas de resposta e seus respectivos parâmetros.

Se a requisição for enviada com algum campo informado abaixo, o valor dele será preenchido automaticamente, se todos os campos forem enviados, a próxima tela a ser mostrada será a de inserir o cartão.

Requisição		
	Parâmetro	Descrição
OPCIONAL	amount	12 dígitos representando o valor, considerando os últimos 2 dígitos como casas decimais. Exemplo: 000000001234 = R$ 12,34
OPCIONAL	transactionDate	Data da transação a ser estornada. Por padrão será o dia corrente. Enviar no formato: “dd/MM/yyyy”
OPCIONAL	cvNumber	Número do CV da transação a ser estornada. É o mesmo campo que o pagamento retorna.
OPCIONAL	originTerminal	Número lógico do terminal que efetuou a transação a ser estornada. Se não for informado, será utilizado o número lógico do terminal que está executando o procedimento.

Resposta		
Quando retorna?	**Parâmetro**	Descrição
SEMPRE	result	Resultado da transação - para mais informações consulte a legenda deste campo
OPCIONAL	resultDetails	Texto com detalhes do retorno
SEMPRE	amount	12 dígitos representando o valor, considerando os últimos 2 dígitos como casas decimais. Exemplo: 000000001234 = R$ 12,34

Exemplo de implementação

```
public class MainActivity extends AppCompatActivity {

    private final int REQUEST_CODE = 1001;
    private final String ARG_RESULT = "result";
    private final String ARG_RESULT_DETAILS = "resultDetails";
    private final String ARG_AMOUNT = "amount";

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Intent intent = new Intent(Intent.ACTION_VIEW,Uri.parse("getnet://pagamento/v1/refund"));
        startActivityForResult(intent, REQUEST_CODE);
    }

    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (REQUEST_CODE == requestCode && RESULT_OK == resultCode) {
            String result = data.getString(ARG_RESULT);
            String resultDetails = data.getString(ARG_RESULT_DETAILS);
            String amount = data.getString(ARG_AMOUNT);
        }
    }
}
```

Reimpressão

O deeplink **getnet://pagamento/v1/reprint** irá reimprimir o último comprovante. Segue a tabela com os parâmetros de resposta

Response		
Quando retorna?	**Chave**	**Descrição**
SEMPRE	result	Resultado da transação - para mais informações consulte a legenda deste campo

Exemplo de implementação

```
public class MainActivity extends AppCompatActivity {

    private final int REQUEST_CODE = 1001;
    private final String ARG_RESULT = "result";

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("getnet://pagamento/v1/reprint"));
        startActivityForResult(intent, REQUEST_CODE);
    }

    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (REQUEST_CODE == requestCode && RESULT_OK == resultCode) {
            String result = data.getString(ARG_RESULT);
        }
    }
}
```

Info

O deeplink **getnet://pagamento/v1/getinfos** irá retornar os dados do terminal via activityResult. Segue a tabela com os parâmetros de resposta

Resposta		
Quando retorna?	**Chave**	**Descrição**
SEMPRE	result	Resultado da transação - para mais informações consulte a legenda deste campo
SEMPRE	ec	Número do estabelecimento comercial vinculado
SEMPRE	numserie	Número de série do equipamento
SEMPRE	numlogic	Número lógico do equipamento vinculado
SEMPRE	version	Versão da API
SEMPRE	cnpjEC	CNPJ do estabelecimento vinculado à Getnet
SEMPRE	nomeEC	Nome do estabelecimento vinculado

Exemplo de implementação

```
public class MainActivity extends AppCompatActivity {

    private final int REQUEST_CODE = 1001;
    private final String ARG_RESULT = "result";
    private final String ARG_EC = "ec";
    private final String ARG_NUM_SERIE = "numserie";
    private final String ARG_NUM_LOGIC = "numlogic";
    private final String ARG_VERSION = "version";
    private final String ARG_CNPJ = "cnpjEC";
    private final String ARG_NAME = "nomeEC";

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("getnet://pagamento/v1/getinfos"));
        startActivityForResult(intent, REQUEST_CODE);
    }

    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (REQUEST_CODE == requestCode && RESULT_OK == resultCode) {
            String result = data.getString(ARG_RESULT);
            String ec = data.getString(ARG_EC);
            String numSerie = data.getString(ARG_NUM_SERIE);
            String numLogic = data.getString(ARG_NUM_LOGIC);
            String apiVersion = data.getString(ARG_VERSION);
            String cnpj = data.getString(ARG_CNPJ);
            String name = data.getString(ARG_NAME);
        }
    }
}
```

SDK

O SDK Getnet é uma camada de abstração entre o app e o hardware disponibilizado através de serviços dentro do Android. Ao utilizar esse SDK, o aplicativo estará sendo suportado por todos os modelos de POS Digital da Getnet. O SDK é composto por 3 arquivos:

servicePosDigital.apk: é o serviço do SDK que fica rodando no terminal que se comunica com os recursos de hardware;

libPosDigital.aar: é a interface do SDK que deve ser compilado com o seu app para se comunicar com o serviço

Devkit.apk: é um app de ajuda que acessa os serviços e tem alguns exemplos. Com ele, o desenvolvedor consegue validar se os serviços de hardware estão funcionando corretamente.

IMPORTANTE: A Getnet se reserva ao direito de solicitar ao desenvolvedor que adapte o seu aplicativo quando houver alguma mudança no SDK. Métodos deprecados serão mantidos somente durante uma versão.

Abaixo estão os serviços disponíveis e como eles estão em cada hardware.

Serviço	Gertec - GPOS700	Ingenico – APOS A8
Card	Suporta	Suporta
Printer	Suporta	Suporta
Mifare	Em construção *	Suporta
Leds	Suporta	Suporta
Beeper	Em construção	Suporta
Camera (Leitor de barcode e qrcode)	Não suporta *	Suporta

* A Getnet recomenda que o desenvolvedor utilize a API direto do Android.

Configurar Gradle

Importar a lib “libposdigital.arr” no Gradle e a classe *PosDigital* estará disponível para uso. No gradle do seu app adicione este trecho:

```
android {
  compileSdkVersion 27
  defaultConfig {
    minSdkVersion 22
    targetSdkVersion 27
  }
  compileOptions {
    sourceCompatibility JavaVersion.VERSION_1_8
    targetCompatibility JavaVersion.VERSION_1_8
  }
}
dependencies {
  implementation fileTree(dir: 'libs', include: ['*.aar'])
}
```

Fazendo bind no service

Adicione no AndroidManifest.xml a permissão abaixo:

```
<uses-permission android:name="com.getnet.posdigital.servi
ce.POSDIGITAL" />
```

A classe *PosDigital* possui métodos que auxiliam no bind do serviço:

```
PosDigital.register(getApplicationContext(), new PosDigita
l.BindCallback(){

  @Override
  public void onError(Exception e) {

  }

  @Override
  public void onDisconnected() {

  }

  @Override
  public void onConnected() {

  }

});
```

Somente após o método onConnected ser executado que os serviços irão funcionar. Para testar se o serviço está conectado execute o método *PosDigital.getInstance().isInitiated().*

Card

Para utilizar os eventos de cartões basta chamar o método PosDigital.getInstance().getCard() e estão disponíveis as funções das leitoras.
Exemplo:

```
int timeout = 30; // Segundos
int[] searchType = {SearchType.MAG, SearchType.CHIP, Searc

hType.NFC};
PosDigital.getInstance().getCard().search(timeout, searchT
ype, new ICardCallback.Stub(){

  @Override
  public void onCard(CardResponse cardResponse) {

  }

  @Override
  public void onMessage(String message) {

  }

  @Override
  public void onError(String error) {

  }

});
```

Impressora

Para utilizar a impressora basta chamar o método *PosDigital.getInstance().getPrinter()* e estão disponíveis as funções da impressora.
Exemplo:

```
PosDigital.getInstance().getPrinter().init();
PosDigital.getInstance().getPrinter().setGray(5);
PosDigital.getInstance().getPrinter().defineFontFormat(Fon
tFormat.MEDIUM);
PosDigital.getInstance().getPrinter().addText(AlignMode.LE
FT, "EXEMPLO DE TEXTO");
PosDigital.getInstance().getPrinter().print(new IPrinterCa
llback.Stub() {
 @Override
 public void onSuccess() {
  // Imprimiu o texto
 }

 @Override
 public void onError(int cause) {
    // Ocorreu Algum erro
 }
});
```

IMPORTANTE: sempre agrupe as informações no comprovante antes de mandar imprimir, ou seja, não chame o método "*print*" toda vez que fazer um addText, addImage, add....

IMPORTANTE: sempre que chamar o método "*print*", aguarde os eventos de call-back antes de chamar outro "*print*".

O tratamento de erros da impressão pode ser feito com o auxílio das constantes da classe *PrinterStatus*.

```
PrinterStatus.OK -> "OK"
PrinterStatus.PRINTING -> "Imprimindo"
PrinterStatus.ERROR_NOT_INIT -> "Impressora não iniciada"
PrinterStatus.ERROR_OVERHEAT -> "Impressora superaquecida"
PrinterStatus.ERROR_BUFOVERFLOW -> "Fila de impressão muit
o grande"
PrinterStatus.ERROR_PARAM -> "Parametros incorretos"
PrinterStatus.ERROR_LIFTHEAD -> "Porta da impressora abert
a"
PrinterStatus.ERROR_LOWTEMP -> "Temperatura baixa demais p
ara impressão"
PrinterStatus.ERROR_LOWVOL -> "Sem bateria suficiente para
 impressão"
PrinterStatus.ERROR_MOTORERR -> "Motor de passo com proble
mas"
PrinterStatus.ERROR_NO_PAPER -> "Sem bonina"
PrinterStatus.ERROR_PAPERENDING -> "Bobina acabando"
PrinterStatus.ERROR_PAPERJAM -> "Bobina travada"
PrinterStatus.UNKNOW -> "Não foi possível definir o erro"
```

Mifare

Para utilizar o mifare basta chamar o método PosDigital.getInstance().getMifare() e estão disponíveis as funções do mifare.

Leds

Para ligar e desligar os Leds basta chamar o método PosDigital.getInstance().getLeds() e estarão disponíveis as funções dos lds.

```
PosDigital.getInstance().getLed().turnOnAll()
PosDigital.getInstance().getLed().turnOffAll()
```

Beeper

Para utilizar o beeper basta chamar o método PosDigital.getInstance().getBeeper() e estão disponíveis as funções do beeper.

```
PosDigital.getInstance().getBeeper().success()
```

Camera

Para utilizar a câmera basta chamar o método PosDigital.getInstance().getCamera() e estão disponíveis as funções da câmera.

```
  int timeout = 30 // segundos
  PosDigital.getInstance().getCamera().readBack(timeout, n
ew ICameraCallback.Stub(){

    @Override
    public void onSuccess(String code){

    }

    @Override
    public void onTimeout(){

    }

    @Override
    public void onCancel(){

    }

    @Override
    public void onError(String error) {
    }

  });
```

Info

Para obter as informações de versão do SDK, do SO, do número de série e da biblioteca compartilhada usar o método:
PosDigital.getInstance().getInfo().info(callbackInfo()).

Como pode ser visto no exemplo abaixo as informações estão disponíveis no *callback* onInfo no objeto InfoResponse.

```
try {
    PosDigital.getInstance().getInfo().info(new IInfoCallback.Stub() {
        @Override
        public void onInfo(InfoResponse infoResponse) throws RemoteException {
            StringBuilder info = new StringBuilder()
                    .append("SDK Version:[").append(infoResponse.getSdkVersion()).append("]\n")
                    .append("BC Version:[").append(infoResponse.getBcVersion()).append("]\n")
                    .append("OS Version:[").append(infoResponse.getOsVersion()).append("]\n")
                    .append("Serial Number:[").append(infoResponse.getSerialNumber()).append("]");
        }

        @Override
        public void onError(String s) throws RemoteException {
            Log.e(TAG, String.format("onError %s",s));
        }
    });
} catch (RemoteException e) {
    e.printStackTrace();
}
```

Estatística

Para consultar as estatísticas do terminal, tais como:

- Consumo de bobina
- Leitura de cartão magnético (tarja)
- Leitura de cartão EMV contato (chip)

- Leitura de cartão EMV sem contato (Contactless)
- Leitura de cartão Mifare

O serviço Getnet possui um conjunto de métodos, divididos em ***estatística geral*** e ***estatística por apps*** e as estatísticas de leitura de cartão possuem funções de leitura com sucesso e falha.

Estatística Geral

```
PosDigital.getInstance().getStatistic().getPaperStatus(IStatCallback callback)
PosDigital.getInstance().getStatistic().getMifareStatus(IStatCallback callback)
PosDigital.getInstance().getStatistic().getMifareStatusFail(IStatCallback callback)
PosDigital.getInstance().getStatistic().getMagStatus(IStatCallback callback)
PosDigital.getInstance().getStatistic().getMagStatusFail(IStatCallback callback)
PosDigital.getInstance().getStatistic().getChipStatus(IStatCallback callback)
PosDigital.getInstance().getStatistic().getChipStatusFail(IStatCallback callback)
PosDigital.getInstance().getStatistic().getNfcStatus(IStatCallback callback)
PosDigital.getInstance().getStatistic().getNfcStatusFail(IStatCallback callback)
```

Estatística por App

```
PosDigital.getInstance().getStatistic().getAllAppPaperStatus(IStatCallback.Stub callback)
PosDigital.getInstance().getStatistic().getAllAppMagStatus(IStatCallback callback)
PosDigital.getInstance().getStatistic().getAllAppMagStatusFail(IStatCallback callback)
PosDigital.getInstance().getStatistic().getAllAppChipStatus(IStatCallback callback)
PosDigital.getInstance().getStatistic().getAllAppChipStatusFail(IStatCallback callback)
PosDigital.getInstance().getStatistic().getAllAppNfcStatus(IStatCallback callback)
PosDigital.getInstance().getStatistic().getAllAppNfcStatusFail(IStatCallback callback)
```

A informação é obtida através do *callback* **IStatCallback**. O exemplo mostra como obter a estatística do consumo de bobina do terminal.

```
PosDigital.getInstance().getStatistic().getPaperStatus( new IStatCallback.Stub() {
        @Override
        public void onStatistic(StatResponse statResponse) throws RemoteException {
            Log.i(TAG, String.format("Consumo de bobina (Geral): %s",
                                     statResponse.getGeneralPaperStatus()));
        }
        @Override
        public void onError(String s) throws RemoteException {
            Log.e(TAG, String.format("Erro: %s", s));
        }
    });
```

Dúvidas Frequentes

1. **A Getnet possui um SDK para abstração de hardware?**
   Ainda não. Este SDK está em construção, e no futuro será disponibilizado. Por enquanto, para usar os recursos do hardware, deve-se verificar diretamente com o fabricante por intermédio da Getnet.
2. **Quem deve imprimir o comprovante da transação de pagamento?**
   A aplicação de pagamento da Getnet. Quando o seu aplicativo chamar o método "StartActivityForResult", a aplicação da Getnet tomará conta da tela do terminal e executará toda transação de pagamento até a impressão do CV e remoção do cartão (em caso de chip). Inclusive mostrará a mensagem na tela de sucesso ou erro. Essa mensagem será retornada para seu aplicativo com resposta da API.
3. **A Getnet pode me enviar o apk de pagamento para eu fazer testes?**
   Não. Por questões de segurança, a aplicação de pagamento da Getnet não pode ser enviada. Por isso criamos o simulador, com ele você pode simular o protocolo e as chamadas da API.
4. **Como vou saber se a integração do meu aplicativo com a aplicação da Getnet está funcionando?**
   A utilização do simulador garante que a API está de acordo, porém somente durante a certificação interna da Getnet que teremos a validação completa. Durante esta certificação, a Getnet utiliza a aplicação de pagamento oficial para os testes de integração.
5. **Toda vez que eu mandar uma versão nova a Getnet irá certificar?**
   Sim, mas o tempo de testes pode variar.

6. **Meu aplicativo ainda não está pronto, mas eu posso iniciar a certificação da Getnet para avaliar a integração com pagamento?**
   Não, pois a Getnet entende que um aplicativo submetido já está validado pelo desenvolvedor e apto para ser publicado para produção. Inclusive, se o apk for aprovado pela certificação, o mesmo será assinado pela Getnet e já estará pronto para produção.
7. **Todo o controle da transação de adquirencia será realizado pela aplicação de pagamento da Getnet? Inclusive a situação das transações pendentes e confirmadas?**

   Sim, a aplicação de pagamento é responsável por toda parte da adquirencia.

8. **Eu terei que colocar o meu simcard 3G/4G no POS?**

   Não, o simcard será da Getnet.

9. **Existe alguma limitação ou requisito para o uso do Wifi?**

   Sim, o POS Digital, por questões de segurança e PCI, não pode se conectar à uma rede wifi sem segurança (sem senha), assim como também não pode entrar em redes que exijam autenticação via browser.

10. **Eu gostaria de fazer a leitura de cartões mifare no meu aplicativo, é possível?**

    Sim, atualmente, nos terminais da Gertec (GPOS700), você pode utilizar o SDK nativo do Android. Para o terminal da Ingenico (APOS A8), é necessário utilizar o SDK.

11. **Qual o contato da Ingenico?**

    BRGDAPOS@ingenico.com

12. **Qual o contato da Gertec?**

    suporte.integracoes@gertec.com.br

13. **A versão do android é 5.1 (Lollipop), basta minha aplicação suportar o Android 5.1 (22) ou necessariamente tenho que compilar para SDK 22?**

    Isso. Basta ter suporte para versão 22.

14. **Posso compilar em um SDK superior e informar a minSDK 22?**

    O target pode ser superior, mas o inSDK deve ser 22.

15. **A máquina que estamos testando veio com o sistema operacional Android na versão 5.1, existe a possibilidade de atualizarmos essa versão para uma mais moderna?**

    Os modelos de terminais que estamos trabalhando hoje são android 5.1 e não temos expectativa de atualizar. Pode acontecer, no futuro, de passarmos a trabalhar com outros modelos de POS com android mais atual.